30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 21

Build System & Gradle — Part 1: Lifecycle, Kotlin DSL, Catalogs & Caching



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 21

Why the build system matters in 2026

Every change you make waits for Gradle. In a 100-module app the build is a product of its own — and interviewers use it to separate people who *use* Android Studio from people who can *own* a codebase.

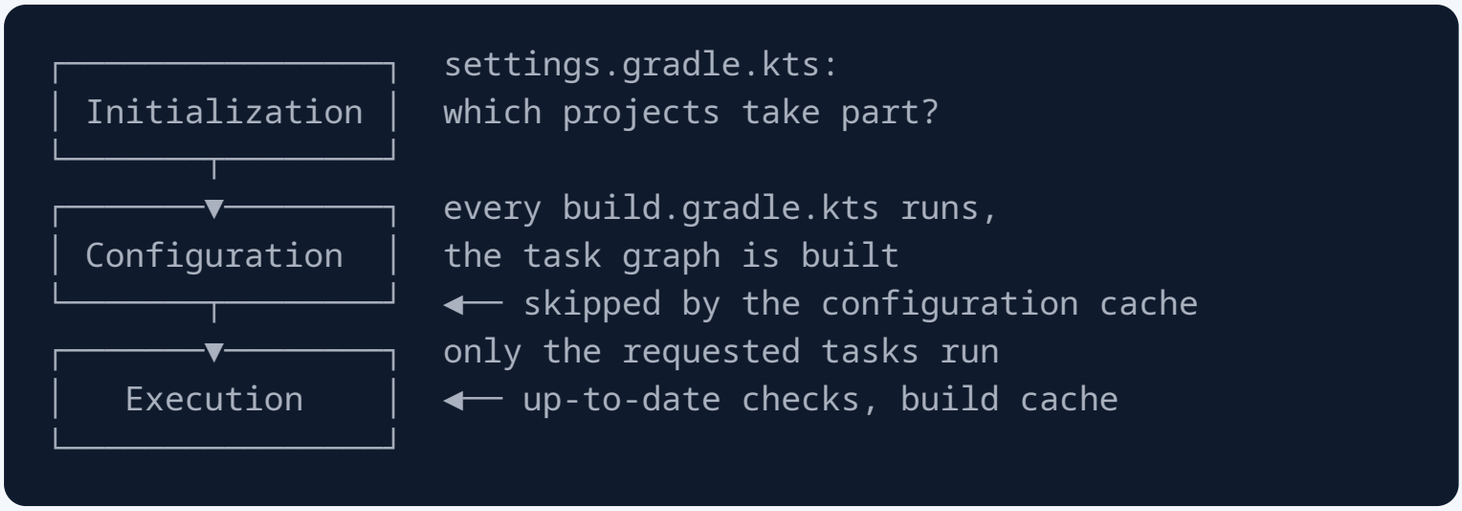
- Build time is team velocity: sync, incremental builds and CI are paid by every engineer, every day
- The modern baseline: **Kotlin DSL, version catalogs, convention plugins** in `build-logic`
- The **configuration cache** is Gradle 9's preferred execution mode; the **build cache** is still opt-in
- kapt is in maintenance mode and doesn't work with AGP 9's built-in Kotlin — **KSP** is the default
- Part 2 (Day 22): dependencies, variants, R8 rules, upgrades, custom tasks

Level	Questions
Junior	From source to APK/AAB · The Gradle files · The build lifecycle
Mid	Kotlin DSL & SDK levels · Version catalogs · KSP vs kapt · Build cache
Senior	Configuration cache · Convention plugins · Tuning `gradle.properties`
Architect	Build logic at scale · Reproducible builds · Modernizing a legacy build



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: three phases, two caches



- **Initialization** reads `settings.gradle.kts` and decides which projects exist in this build
- **Configuration** runs the build scripts of those projects and builds the task graph — on *every* build, unless the configuration cache is reused
- **Execution** runs only the tasks needed for what you asked, skipping tasks whose inputs didn't change

TWO CACHES, TWO PHASES

The configuration cache skips **configuration**. The build cache skips **task execution**. Interviewers love to check that you know the difference.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 21

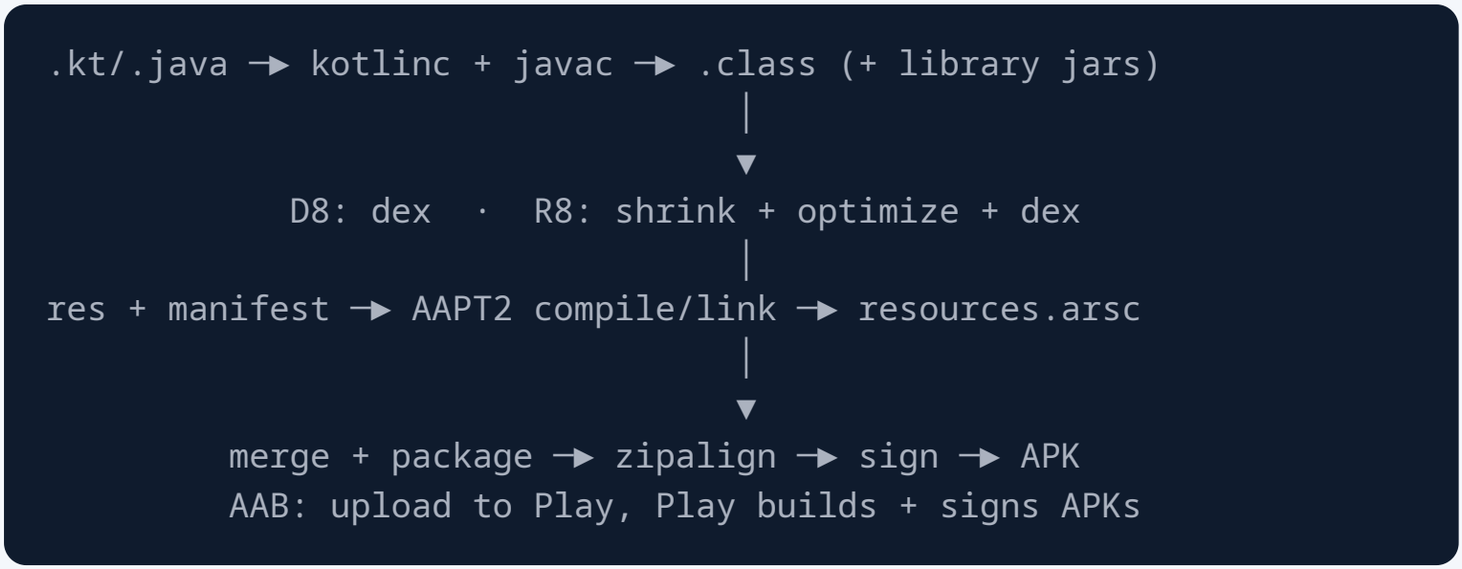
Junior

From source to APK/AAB · The Gradle files · The build lifecycle



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. From source code to an installable app



- **Gradle** runs the build; the **Android Gradle Plugin (AGP)** adds the Android tasks
- Kotlin and Java compile to JVM bytecode; **D8** turns it into DEX for the Android runtime
- **AAPT2** compiles resources and links them with the manifest into `resources.arsc` and the `R` class



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. The details interviewers probe

- `R` is generated directly as bytecode — there is no `R.java` to edit. Since AGP 8.0, `R` classes are **non-transitive**: a module's `R` contains only its own resources.
- D8 also **desugars** newer Java APIs for older devices. With `isMinifyEnabled = true`, **R8** replaces it and shrinks, optimizes, obfuscates and dexes in one pass.
- Google Play requires an **AAB** for new apps. With Play App Signing you sign the bundle with an *upload key*; Play generates the APKs and signs them with the app signing key.

INTERVIEW TRAP

"Sign the APK, then zipalign it." — Backwards. With `apksigner`, zipalign runs **before** signing: changing the file afterwards invalidates the v2+ signature. AGP does both, in the right order, for you.

FOLLOW-UP

"What does zipalign actually do?" — It aligns uncompressed data so the system can memory-map it straight from the APK. Uncompressed native libraries must be page-aligned — that's why 16 KB alignment matters for apps with native code.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. The Gradle files and what each one is for

File	What it's for
`settings.gradle.kts`	Which modules exist, plugin and dependency repositories, included builds
`build.gradle.kts` (root)	Declares plugins with `apply false` — ideally nothing else
`app/build.gradle.kts`	Applies plugins; `android {}` config; dependencies
`gradle/libs.versions.toml`	Version catalog: libraries, plugins, versions
`gradle.properties`	Flags for Gradle, the JVM, Kotlin and AGP
`gradle/wrapper/gradle-wrapper.properties`	The exact Gradle version everyone gets via `./gradlew`

INTERVIEW TRAP

"The root build file holds `buildscript { classpath … }` and `allprojects { repositories {} }`." — That's the legacy layout. Today plugins are declared in `plugins {}` blocks with catalog aliases, and repositories live in one place: `settings.gradle.kts`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. A modern `settings.gradle.kts`

```
pluginManagement {
    includeBuild("build-logic")
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}
rootProject.name = "Shop"
include(":app", ":core:data", ":feature:cart")
```

- `FAIL_ON_PROJECT_REPOS`: a module that declares its own repository fails the build — one place to audit where artifacts come from
- `includeBuild("build-logic")` makes your convention plugins (Q9) available by ID



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. The build lifecycle in code

```
// settings.gradle.kts — INITIALIZATION
include(":app", ":core:data")

// app/build.gradle.kts — CONFIGURATION (every build)
println("Configuring :app")

tasks.register("hello") {
    // Configuration action: runs only if 'hello' is needed
    group = "demo"
    doLast {
        // EXECUTION: runs only when 'hello' executes
        println("Hello from the execution phase")
    }
}
```

- `./gradlew hello` prints both lines; `./gradlew help` still prints *"Configuring :app"*
- Only task actions — `doFirst`, `doLast`, `@TaskAction` — run in the execution phase



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. What interviewers check

INTERVIEW TRAP

"The code inside `tasks.register { }` runs when the task runs." — No. That block **configures** the task. Only its actions (`doFirst` / `doLast`) run at execution time.

INTERVIEW TRAP

Doing real work during configuration — reading files, calling `git`, hitting the network — slows down **every** build and every Android Studio sync, even when nobody runs that task.

FOLLOW-UP

"Why is Gradle sync slow?" — Sync is dominated by configuration and dependency resolution. Every piece of configuration-time work shows up directly as sync time.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 21

→

Mid-Level

Kotlin DSL & SDK levels · Version catalogs · KSP vs kapt · Build cache



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. A module build file in Kotlin DSL

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
}

android {
    namespace = "com.example.shop"
    compileSdk = 36
    defaultConfig {
        applicationId = "com.example.shop"
        minSdk = 24
        targetSdk = 36
    }
}

dependencies {
    implementation(libs.androidx.core.ktx)
}
```

- Kotlin DSL is the default for new Android Studio projects: compile-time checks, completion, navigation into the DSL
- With **AGP 9**, Kotlin support is built in — no `org.jetbrains.kotlin.android` plugin. The Compose compiler plugin is still applied.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. SDK levels — and a Kotlin DSL trap

Setting	What it means
`compileSdk`	Which Android APIs you can **compile** against. No runtime behavior change
`minSdk`	Lowest Android version that can install the app; Lint flags newer API calls
`targetSdk`	Which version's **behavior changes** you opt into. Play requires a recent one

INTERVIEW TRAP

"Just set targetSdk to the latest." — Raising it is a **migration**, not a number: targeting 35 enforces edge-to-edge; targeting 36 stops calling `onBackPressed()`. Read the behavior changes and test first.

INTERVIEW TRAP

Type-safe accessors — `android {}`, `implementation(…)` — exist only for plugins applied in the `plugins {}` block. With `apply(plugin = "…")` the script fails with *"unresolved reference"*.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Version catalogs

```toml
# gradle/libs.versions.toml
[versions]
agp = "9.1.0"
ksp = "2.3.11"
room = "2.7.2"

[libraries]
room-runtime = { module = "androidx.room:room-runtime", version.ref = "room" }
room-compiler = { module = "androidx.room:room-compiler", version.ref = "room" }
junit = "junit:junit:4.13.2"

[plugins]
android-library = { id = "com.android.library", version.ref = "agp" }
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

- `-`, `_` and `.` in an alias become `.` in the accessor: `room-runtime` → `libs.room.runtime`
- Plugin versions are declared once; modules apply them by alias, without versions



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Using the catalog

```
plugins {
    alias(libs.plugins.android.library)
    alias(libs.plugins.ksp)
}

dependencies {
    implementation(libs.room.runtime)
    ksp(libs.room.compiler)
    testImplementation(libs.junit)
}
```

- One file to review, and one file for update bots such as Renovate to edit
- It replaces the old `buildSrc/Dependencies.kt` constants and versions stored in `gradle.properties`

INTERVIEW TRAP

"The catalog pins versions for the whole app." — No. It's a list of names and **requested** versions. Conflict resolution can still pick a higher version brought in transitively. Aligning and enforcing versions needs platforms, BOMs and constraints — Day 22.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. KSP vs kapt

	kapt	KSP
How it works	Generates Java stubs of your Kotlin code, then runs Java annotation processors	Reads Kotlin symbols directly — no stubs
Speed	Stub generation + javac on every change	Officially "up to 2× faster"
Kotlin awareness	Sees a Java view of your code	Understands nullability and Kotlin constructs
Status	Maintenance mode; `kotlin-kapt` is incompatible with AGP 9's built-in Kotlin	The standard: Room, Dagger/Hilt, Moshi, Glide
Multiplatform	JVM only	Works in KMP projects

INTERVIEW TRAP

"KSP is just a faster kapt." — It's a different API. Java annotation processors don't run on KSP; each library must ship a KSP processor. Data Binding never will — keep it isolated in its own modules or move to View Binding.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Migrating, module by module

```
plugins {
    alias(libs.plugins.android.library)
    alias(libs.plugins.ksp)
    alias(libs.plugins.hilt)
}

dependencies {
    implementation(libs.room.runtime)
    ksp(libs.room.compiler)       // was: kapt(…)
    implementation(libs.hilt.android)
    ksp(libs.hilt.compiler)
}
```

- Most libraries ship the KSP processor in the same artifact: `kapt(` becomes `ksp(`
- On AGP 9, a module that can't drop kapt yet uses the `com.android.legacy-kapt` plugin as a temporary bridge
- Since KSP 2.3.0, the KSP version is no longer tied to the Kotlin version

INTERVIEW TRAP

Keeping kapt and KSP side by side in one module "because it works" — while **any** kapt processor remains, the module still generates stubs. The big win only comes when the last kapt usage leaves the module.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Up-to-date checks vs the build cache

Task outcome	Meaning
`UP-TO-DATE`	Inputs and outputs unchanged since the last run **in this workspace**
`FROM-CACHE`	Outputs restored from the build cache — produced earlier, on another branch or machine
`NO-SOURCE`	Nothing to process
(no label)	The task really executed

- Both depend on **declared inputs and outputs**. The cache key is a hash of the task's implementation and all its inputs.
- The build cache is **off by default**: `org.gradle.caching=true` (or `--build-cache`)
- The local cache lives in `~/.gradle/caches/build-cache-1`; a shared remote cache is Day 22



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. The build cache in action

```
$ ./gradlew clean :app:assembleDebug
> Task :core:data:compileDebugKotlin FROM-CACHE
> Task :feature:cart:compileDebugKotlin FROM-CACHE
> Task :app:compileDebugKotlin
…
BUILD SUCCESSFUL
312 actionable tasks: 41 executed, 271 from cache
```

INTERVIEW TRAP

"The local build cache is enabled by default." — It isn't (last year's version of this challenge got this wrong). Without `org.gradle.caching=true`, task outputs are never stored or reused.

INTERVIEW TRAP

"`clean` makes caching useless." — The opposite: after `clean`, cacheable tasks come back `FROM-CACHE`. What `clean` destroys is only the up-to-date state in the `build/` folders.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 21



Senior

Configuration cache · Convention plugins · Tuning gradle.properties



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. The configuration cache

It stores the **result of the configuration phase** — the task graph and every task's inputs — and reuses it while nothing that affected configuration changed: build scripts, `gradle.properties`, environment variables or files read during configuration.

```
# gradle.properties
org.gradle.configuration-cache=true
# while migrating only: report problems instead of failing
org.gradle.configuration-cache.problems=warn
```

	Configuration cache	Build cache
Skips	Configuration (scripts, task graph)	Task execution
Shared across machines	No — local to the build	Yes, with a remote cache
Status	Stable since Gradle 8.1; preferred mode in Gradle 9, still opt-in	Opt-in

With the configuration cache on, tasks also run **in parallel inside the same project**, not only across modules.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. What breaks it — and the fix

```
// ❌ Uses Project at execution time
tasks.register("printVersion") {
    doLast { println(project.version) }
}

// ✅ Capture the value during configuration
tasks.register("printVersion") {
    val version = project.version.toString()
    doLast { println(version) }
}

// ✅ Read the environment through providers
val isCi = providers.environmentVariable("CI").isPresent
```

- Also incompatible: build listeners like `gradle.buildFinished {}` and `TaskExecutionListener`, and plugins that haven't been updated
- The HTML report under `build/reports/configuration-cache/` lists every problem

INTERVIEW TRAP

"We run with `problems=warn`, so we're compatible." — Warning mode is a migration aid. It hides problems that can produce incorrect builds; Gradle's docs say not to keep it as a permanent setting.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Convention plugins in `build-logic`

Instead of copying the same `android {}` block into 40 modules, write it once as a plugin and apply it by ID.

```
build-logic/
├── settings.gradle.kts    reuses ../gradle/libs.versions.toml
└── convention/
    ├── build.gradle.kts   kotlin-dsl + gradlePlugin {}
    └── src/main/kotlin/AndroidLibraryConventionPlugin.kt
```

```
// build-logic/convention/build.gradle.kts
plugins { `kotlin-dsl` }

dependencies {
    compileOnly(libs.android.gradlePlugin)
}

gradlePlugin {
    plugins {
        register("androidLibrary") {
            id = "shop.android.library"
            implementationClass = "AndroidLibraryConventionPlugin"
        }
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. The plugin and its use

```
class AndroidLibraryConventionPlugin : Plugin<Project> {
    override fun apply(target: Project) {
        with(target) {
            pluginManager.apply("com.android.library")
            extensions.configure<LibraryExtension> {
                compileSdk = 36
                defaultConfig.minSdk = 24
            }
            dependencies {
                val junit = libs.findLibrary("junit").get()
                add("testImplementation", junit)
            }
        }
    }
}

// core/data/build.gradle.kts
plugins { id("shop.android.library") }
```

- `LibraryExtension` is the new DSL type from `com.android.build.api.dsl` — the only one AGP 9 exposes
- Generated `libs.x` accessors don't exist in plugin classes; `libs` here is a small extension over `VersionCatalogsExtension`

INTERVIEW TRAP

"*`allprojects {}` in the root script is simpler.*" — Cross-project configuration couples every module to the root, hides where settings come from, and blocks Gradle's Isolated Projects. Convention plugins are opt-in, per module.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Tuning `gradle.properties`

```
org.gradle.jvmargs=-Xmx4g -XX:+UseParallelGC -Dfile.encoding=UTF-8
kotlin.daemon.jvmargs=-Xmx3g
org.gradle.parallel=true
org.gradle.caching=true
org.gradle.configuration-cache=true
```

Property	Effect	Default
`org.gradle.jvmargs`	Heap and GC of the Gradle daemon, where AGP's tasks run	Small — set it
`kotlin.daemon.jvmargs`	Heap of the separate Kotlin compiler daemon	Inherited
`org.gradle.parallel`	Runs tasks of different projects in parallel	Off
`org.gradle.caching`	Build cache	Off
`org.gradle.configuration-cache`	Configuration cache	Off

Machine-specific values (heap size) can go in `~/.gradle/gradle.properties`, which overrides the project file.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Myths about build speed

INTERVIEW TRAP

"Use `--offline` to make builds faster." — With fixed versions, Gradle already uses its local dependency cache and doesn't hit the network. What forces network checks are **dynamic** (`1.+`) and `SNAPSHOT` versions — remove those instead.

INTERVIEW TRAP

"More heap is always faster." — Heap beyond what the build needs doesn't speed it up and takes memory from Android Studio and the emulator. Check GC time in a build scan before and after — Day 22.

WHAT ACTUALLY MOVES THE NEEDLE

Configuration cache, build cache, no kapt, lazy task APIs, and a module graph with small recompile scope (`implementation` over `api` — Day 22). Flags only unlock what the build's design allows.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 21

Architect

Build logic at scale · Reproducible builds · Modernizing a legacy build



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Build logic as a product

```
shop.jvm.library           pure Kotlin modules (domain)
shop.android.library       compileSdk, minSdk, Lint, tests
shop.android.compose       Compose compiler plugin + BOM
shop.android.feature       library + compose + DI + :core:ui
shop.android.application   signing, R8, versioning
```

- **Compose plugins, don't inherit:** the feature plugin applies the library and compose plugins; each concern lives in exactly one place
- A module's build file says **what it is** (plugin IDs) and **what it needs** (dependencies) — nothing else
- Test plugins like production code, with Gradle TestKit (`GradleRunner`)

INTERVIEW TRAP

"buildSrc is deprecated — you must use build-logic." — Neither is deprecated; both can host convention plugins. An included `build-logic` build (the Now in Android setup) is more explicit: nothing lands on every build script's classpath implicitly — modules opt in by plugin ID.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Keep configuration lazy

```
// ❌ Eager: creates and configures every Test task now
tasks.withType<Test> {
    maxHeapSize = "1g"
}

// ✅ Lazy: configures only tasks that end up in the graph
tasks.withType<Test>().configureEach {
    maxHeapSize = "1g"
}

// ✅ Lazy lookup by name
tasks.named<Test>("testDebugUnitTest") {
    useJUnitPlatform()
}
```

- Configuration time grows with every module — plugin code runs once per project, on every build without a configuration-cache hit, and on every sync
- No reaching into other projects (`project(":x").extensions…`) and no `afterEvaluate` ordering hacks — wire values with lazy `Provider`s instead

INTERVIEW TRAP

"*`tasks.withType<T> { }` is the lazy API.*" — The lambda overload realizes **every** task of that type. Only `withType<T>().configureEach { }` defers the work.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Reproducible builds: same inputs, same result, any machine

Risk	Control
Different Gradle versions	Commit the wrapper, set `distributionSha256Sum`, validate the wrapper jar in CI
Different JDKs	A Java toolchain: `kotlin { jvmToolchain(17) }` — compilation no longer depends on `JAVA_HOME`
Tampered or swapped artifacts	Dependency verification: `gradle/verification-metadata.xml`
Moving versions	No dynamic or `SNAPSHOT` versions; dependency locking if you must use ranges
Non-deterministic outputs	No timestamps or machine paths in generated code or resources

INTERVIEW TRAP

"It builds on my machine, so the JDK is fine." — The JDK that **runs** Gradle (Android Studio's Gradle JDK or `JAVA_HOME`) and the toolchain that **compiles** your code are separate settings. Gradle 9 and AGP 9 both need JDK 17+ to run.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. In practice

```
# Pin Gradle and its checksum for everyone
./gradlew wrapper --gradle-version 9.1.0 \
    --gradle-distribution-sha256-sum <sha256>

# Record checksums of every dependency and plugin
./gradlew --write-verification-metadata sha256 help
```

```
// ❌ A new value on every build
buildConfigField(
    "long", "BUILD_TIME", "${System.currentTimeMillis()}L"
)
```

INTERVIEW TRAP

A timestamp or git SHA in the **debug** `BuildConfig` changes an input on every build: the module recompiles and everything downstream misses the cache. Inject such values only in release/CI builds, or read them at runtime.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Modernizing a slow legacy build

Scenario: sync is slow, CI is slow, and 60 modules each have their own copy-pasted build file. In what order do you fix it?

#	Step	Why here
1	Measure a baseline: build scans, Build Analyzer	You can't prove a win without numbers — Day 22
2	Upgrade Gradle, AGP and Kotlin	Performance fixes; prerequisites for the rest
3	Version catalog, then convention plugins	Every later change becomes a one-file change
4	Build cache, then configuration cache (`warn` → fix → strict)	Biggest wins for local and CI builds
5	kapt → KSP, module by module	Removes stub generation
6	Split hot modules; `implementation` over `api`	Smaller recompile scope — Days 22 and 25



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. How to run the migration

INTERVIEW TRAP

The big-bang PR — Groovy to Kotlin DSL, buildSrc to build-logic, kapt to KSP and an AGP major, all at once. Nobody can review it, bisect it, or say which change caused a regression. One change per PR, each one measured.

INTERVIEW TRAP

"Converting to Kotlin DSL will speed up the build." — It won't by itself. Kotlin DSL is about correctness and maintainability; speed comes from caching, lazy configuration and the module graph.

FOLLOW-UP

"How do you keep it fast after the project?" — Treat build time as a budget: track configuration time and cache hit rate in CI and alert on regressions (Days 22 and 28).

SENIOR ANSWER IN ONE LINE

"A fast build is designed, not tuned: lazy configuration, cacheable tasks, small modules, and one place for build logic."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 21 · ARCHITECT

Day 21 — cheat sheet

- Pipeline: kotlinc/javac → D8/R8 → DEX; AAPT2 → resources; package → zipalign **then** sign. Play gets an AAB.
- Three phases: initialization (which projects), configuration (scripts + task graph), execution (only the needed tasks).
- No real work at configuration time; task work goes in `doLast` / `@TaskAction`.
- Kotlin DSL + `plugins {}` + version catalog is the baseline. The catalog **names** versions — it doesn't enforce them.
- `compileSdk` = APIs you compile against; `targetSdk` = behavior you opt into. Raising it is a migration.
- KSP over kapt. One kapt processor left in a module keeps the stub cost; AGP 9 needs `legacy-kapt` for leftovers.
- `UP-TO-DATE` ≠ `FROM-CACHE`. The build cache is opt-in: `org.gradle.caching=true`.
- Configuration cache skips configuration; build cache skips execution. Never touch `project` at execution time.
- Convention plugins in `build-logic`, applied by ID. No `allprojects {}`; `configureEach` over eager APIs.
- Reproducible: wrapper + checksum, JDK toolchain, dependency verification, no timestamps in inputs.

Day 22 is next → Gradle part 2: dependencies, variants, R8 rules & upgrades. Save this post and follow along.



Esmaeil Moradi
@esmaeeil-moradi700